



**WYDZIAŁ MATEMATYKI
i INFORMATYKI**

Uniwersytet Łódzki

Artem Hnap

Nr albumu: 417186

**Implementacja sieciowej platformy mini gier
wieloosobowych w silniku Unity z
wykorzystaniem Photon Fusion oraz integracji
Steam SDK**

Praca licencjacka na kierunku Informatyka

Praca wykonana pod kierunkiem prof.

dr Roberta Plebaniaka

Katedra Analizy Nieliniowej

Łódź, 2026

Słowa kluczowe: Unity, Multiplayer, Gry, Photon Fusion, C#, Synchronizacja sieciowa

Title in English: Implementation of a Multiplayer Mini-Game Networking Platform in the Unity Engine Using Photon Fusion a`nd Steam SDK Integration

Keywords: Unity, Multiplayer, Games, Photon Fusion, C#, Network Synchronization

Spis treści

WSTĘP	5
ZAŁOŻENIA FUNKCJONALNE APLIKACJI	7
2.1 Ogólna charakterystyka systemu	7
2.2 Sterowanie postacią gracza	7
2.3 System lobby i zarządzania sesją	7
2.4 System gotowości graczy	8
2.5 System rozgrywki minigier	8
2.6 System punktacji i tablicy wyników	8
WYKORZYSTYWANE TECHONOLOGIE	9
3.1 Środowisko Unity i język programowania C#	9
3.2 Wykorzystane frameworki i biblioteki pomocnicze	9
3.2.1 Photon Fusion	10
3.2.2 Steam SDK	10
ROZWIĄZYWANE PROBLEMY IMPLEMENTACYJNE	11
4.1 Połączenie graczy i synchronizacja ruchu	11
4.2 Stworzenie postaci finalnej i synchronizacja animacji w sieci	13
4.3 Implementacja systemu lobby publicznych i prywatnych	15
4.4 Integracja zaproszeń Steam (Steam Invites)	17
4.5 Implementacja systemu gotowości graczy	19
4.6 Obsługa różnorodnych mini gier	20
4.6.1 Klasa bazowa minigry i cykl życia rozgrywki	21
4.6.2 Konfiguracja minigier przez ScriptableObject	22
4.6.3 Rozszerzanie minigier o różne modele rozgrywki	22
4.6.4 Sceny i spawnowanie graczy	23
4.7 Zarządzanie główną pętlą gry	24
4.8 Tabela liderów i wyniki	26

OPIS APLIKACJI	27
MOŻLIWE KIERUNKI ROZWOJU APLIKACJI	32
BIBLIOGRAFIA	34

Rozdział 1

Wstęp

Rozwój technologii informatycznych oraz powszechny dostęp do szybkich połączeń internetowych sprawił, że gry komputerowe stały się jedną z najdynamiczniej rozwijających się gałęzi przemysłu rozrywkowego. Szczególnie dużą popularność zdobyły gry wieloosobowe, które umożliwiają wspólną rozgrywkę użytkownikom znajdującym się w różnych lokalizacjach.

Implementacja tego typu gier jest znacznie bardziej złożona niż w przypadku gier jednoosobowych. Oprócz standardowych elementów takich jak logika gry, interfejs użytkownika czy obsługa fizyki, konieczne jest zapewnienie poprawnej synchronizacji danych pomiędzy wszystkimi uczestnikami rozgrywki. Wymaga to zastosowania odpowiednich modeli komunikacji sieciowej oraz mechanizmów minimalizujących opóźnienia i zapewniających spójność stanu gry.

Naszym podejściem do projektowania gry wieloosobowej jest stworzenie platformy zawierającej zestaw krótkich minigier, które mogą być rozgrywane przez wielu graczy w ramach jednej sesji. Tego typu rozwiązanie pozwala na utrzymanie dynamicznego tempa rozgrywki oraz zwiększa różnorodność doświadczenia graczy poprzez zmianę mechanik pomiędzy kolejnymi etapami gry. Platformy minigier są często wykorzystywane w grach imprezowych, w których uczestnicy rywalizują ze sobą w serii krótkich konkurencji, a ostateczny wynik zależy od sumy zdobytych punktów.

Współczesne gry komputerowe są jednak nie tylko samodzielnymi aplikacjami, ale również częścią większego ekosystemu usług cyfrowych. Platformy dystrybucji gier, takie jak Steam, oferują szereg funkcjonalności wspierających zarówno twórców, jak i użytkowników gier. Oprócz możliwości zakupu i instalacji gier zapewniają one również funkcje społecznościowe, takie jak system znajomych, zaproszenia do gry, profile użytkowników czy system osiągnięć. Integracja aplikacji z takimi platformami pozwala na łatwiejsze nawiązywanie rozgrywek wieloosobowych oraz poprawia dostępność gry dla użytkowników.

Celem niniejszej pracy jest samodzielna implementacja sieciowej platformy minigier wieloosobowych w silniku Unity z wykorzystaniem biblioteki Photon Fusion oraz integracji z platformą Steam przy użyciu Steam SDK. Całość zaprojektowanego i zrealizowanego systemu – obejmująca logikę rozgrywki, zestaw minigier oraz integrację z zewnętrznymi platformami – stanowi autorski wkład własny.

Praca została podzielona na kilka rozdziałów. Zaczyna się ona od przedstawienia założeń funkcjonalnych projektowanej aplikacji. W kolejnych częściach opisano technologie wykorzystane w procesie implementacji, problemy napotkane podczas tworzenia systemu oraz sposoby ich rozwiązania, a także szczegółowy opis działania aplikacji i jej architektury. Ostatni rozdział zawiera możliwe kierunki dalszego rozwoju projektu.

Realizacja tego typu projektu stanowi interesujące zagadnienie z punktu widzenia inżynierii oprogramowania, ponieważ wymaga samodzielnego połączenia wielu różnych technologii oraz rozwiązań architektonicznych. Dodatkowym aspektem podjętym w ramach pracy jest integracja aplikacji z zewnętrznymi platformami usługowymi, co wymagało wykorzystania udostępnionych interfejsów programistycznych oraz dostosowania architektury aplikacji do współpracy z zewnętrzną infrastrukturą.

Rozdział 2

Założenia funkcjonalne aplikacji

2.1 Ogólna charakterystyka systemu

Projektowana aplikacja musi stanowić sieciową platformę minigier wieloosobowych, umożliwiającą jednoczesną rozgrywkę wielu użytkowników w ramach jednej sesji gry. Głównym założeniem systemu jest zapewnienie płynnej rozgrywki opartej na cyklu kolejnych minigier, pomiędzy którymi prezentowane są wyniki uczestników.

System musi być zaprojektowany w sposób umożliwiający łatwe tworzenie i zarządzanie sesjami gry oraz zapewniać spójne doświadczenie użytkownika niezależnie od liczby uczestników z wybranego przedziału (w naszym przypadku: od 2 do 6 graczy).

2.2 Sterowanie postacią gracza

Aplikacja musi wykorzystywać model sterowania postacią z perspektywy trzeciej osoby. Gracz steruje swoją postacią przy użyciu klawiatury i myszy. Sterowanie musi obejmować poruszanie się w przestrzeni trójwymiarowej, zmianę kierunku widoku oraz wykonywanie akcji zależnych od aktualnej minigry.

2.3 System lobby i zarządzania sesją

Aplikacja musi umożliwiać tworzenie oraz zarządzanie sesjami gry w postaci lobby. Użytkownik musi mieć możliwość utworzenia nowej sesji lub dołączenia do już istniejącej.

Dołączenie do lobby prywatnego musi odbywać się poprzez wprowadzenie wygenerowanego kodu, natomiast do lobby publicznego możliwe jest dołączenie zarówno poprzez kod, jak i poprzez mechanizm losowego dopasowania.

Dodatkowo aplikacja musi umożliwiać dołączanie do sesji gry za pomocą zaproszeń wysyłanych przez platformę Steam. Funkcjonalność ta ma umożliwiać szybkie zapraszanie znajomych do wspólnej rozgrywki bez konieczności ręcznego wprowadzania kodu sesji.

2.4 System gotowości graczy

Przed rozpoczęciem właściwej rozgrywki wszyscy uczestnicy sesji muszą potwierdzić swoją gotowość. System musi umożliwiać każdemu graczowi oznaczenie swojego statusu jako „gotowy”. Aktualny stan gotowości wszystkich uczestników musi być widoczny dla każdego gracza znajdującego się w lobby, co pozwala na kontrolę przygotowania do rozpoczęcia rozgrywki. Rozpoczęcie gry musi następować dopiero w momencie, gdy wszyscy gracze potwierdzą swoją gotowość, co zapobiega startowi w niepełnym składzie.

2.5 System rozgrywki minigier

Rozgrywka w aplikacji musi opierać się na cyklu kolejnych minigier uruchamianych jedna po drugiej w ramach jednej sesji gry. Każda minigra stanowi oddzielny etap rozgrywki i może posiadać własne zasady oraz mechaniki.

System musi zarządzać przechodzeniem pomiędzy kolejnymi minigrami, zapewniając płynne przejścia oraz zachowanie ciągłości rozgrywki. Przed rozpoczęciem każdej minigry mogą być wyświetlane informacje dotyczące jej zasad.

Różnorodność minigier stanowi istotny element aplikacji, dlatego system musi umożliwiać implementację różnych mechanik rozgrywki w zależności od charakteru danej minigry.

2.6 System punktacji i tablicy wyników

Po każdej minigrze ma być wyświetlana aktualna tablica wyników, natomiast po zakończeniu całej sesji prezentowany musi być końcowy ranking wszystkich uczestników.

Rozdział 3

Wykorzystywane technologie

Realizacja projektu sieciowej platformy minigier wieloosobowych wymagała zastosowania odpowiednich technologii umożliwiających implementację logiki gry, obsługę komunikacji sieciowej oraz integrację z zewnętrznymi usługami. Wybór narzędzi i bibliotek wykorzystanych w projekcie został podyktowany ich funkcjonalnością, popularnością w środowisku twórców gier oraz możliwością integracji z silnikiem Unity.

W niniejszym rozdziale przedstawiono technologie wykorzystane podczas realizacji projektu, obejmujące silnik Unity jako główne środowisko tworzenia gry, język programowania C#, bibliotekę Photon Fusion służącą do implementacji rozgrywki wieloosobowej, Steam SDK umożliwiające integrację aplikacji z platformą Steam oraz system kontroli wersji Git.

3.1 Środowisko Unity i język programowania C#

W projekcie jako główne środowisko deweloperskie wykorzystano silnik Unity, który jest jednym z najpopularniejszych narzędzi do tworzenia gier komputerowych. Jego popularność wynika z dużej elastyczności oraz możliwości publikacji na wiele platform, takich jak komputery osobiste, konsole czy urządzenia mobilne.

Silnik Unity oparty jest na języku programowania C#. W projekcie C# został wykorzystany do implementacji wszystkich kluczowych systemów gry.

3.2 Wykorzystane frameworki i biblioteki pomocnicze

W celu usprawnienia procesu tworzenia aplikacji oraz zwiększenia czytelności i modularności kodu wykorzystano dodatkowe frameworki i biblioteki rozszerzające możliwości silnika Unity. Pozwoliły one uprościć implementację wybranych mechanizmów oraz poprawić organizację architektury projektu.

Jednym z użytych narzędzi jest Zenject, który implementuje mechanizm wstrzykiwania zależności (Dependency Injection). Umożliwia on luźniejsze powiązanie komponentów oraz łatwiejsze zarządzanie zależnościami w systemie, co przekłada się na bardziej modułną strukturę kodu.

Do tworzenia animacji i płynnych przejść wykorzystano DoTween, który umożliwia prostą interpolację wartości w czasie. Biblioteka ta została użyta głównie do animacji interfejsu oraz efektów wizualnych, upraszczając implementację animacji w kodzie.

3.2.1 Photon Fusion

Photon Fusion to biblioteka sieciowa przeznaczona do tworzenia gier wieloosobowych w Unity. Umożliwia synchronizację stanu gry w czasie rzeczywistym oraz komunikację pomiędzy graczami przy zachowaniu spójności danych i niskich opóźnień. W projekcie została wykorzystana jako podstawowy system odpowiedzialny za rozgrywkę sieciową.

3.2.2 Steam SDK

Steam SDK jest zestawem narzędzi udostępnianym przez platformę Steam, umożliwiającym integrację aplikacji z funkcjami oferowanymi przez tę platformę. Steam jest jedną z największych platform dystrybucji gier komputerowych i zapewnia funkcjonalności takie jak system kont użytkowników, lista znajomych, zaproszenia do gry, system osiągnięć oraz inne mechanizmy społecznościowe.

W projekcie Steam SDK zostało wykorzystane przede wszystkim do identyfikacji użytkowników oraz integracji z systemem znajomych, co zwiększa wygodę korzystania z aplikacji i ułatwia organizowanie rozgrywek sieciowych.

Rozdział 4

Rozwiązywane problemy implementacyjne

W trakcie realizacji projektu napotkano szereg problemów związanych z implementacją systemu wieloosobowego, synchronizacją danych oraz zarządzaniem przebiegiem rozgrywki. Ze względu na złożoność aplikacji oraz brak wcześniejszego doświadczenia w tworzeniu gier sieciowych, część zagadnień wymagała iteracyjnego podejścia oraz stopniowego rozwijania kolejnych elementów systemu.

W niniejszym rozdziale przedstawiono wybrane problemy implementacyjne wraz z zastosowanymi rozwiązaniami, które umożliwiły osiągnięcie poprawnego działania platformy minigier wieloosobowych.

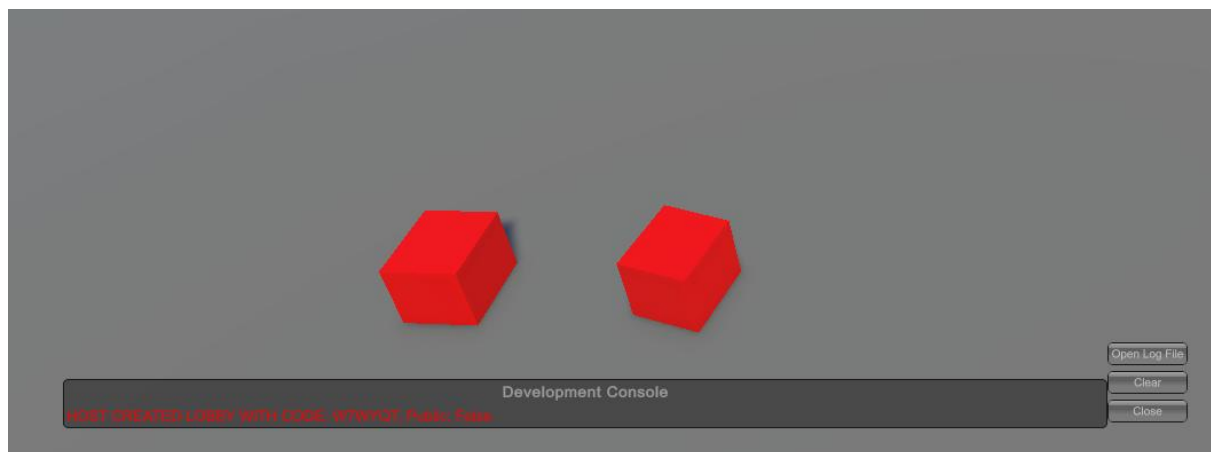
4.1 Połączenie graczy i synchronizacja ruchu

Podstawowym i najważniejszym krokiem w realizacji platformy wieloosobowej było umożliwienie graczom dołączania do wspólnej sesji oraz zapewnienie mechanizmu ich wzajemnej synchronizacji. W tym celu wykorzystano wyżej omówianą bibliotekę Photon Fusion, która dostarcza gotowe rozwiązania do zarządzania sesjami oraz komunikacji sieciowej. Proces połączenia graczy realizowany jest przy użyciu komponentu **NetworkRunner**, który stanowi centralny element systemu sieciowego i odpowiada za uruchomienie oraz obsługę symulacji. Każda instancja gry tworzy własny NetworkRunner, a następnie inicjalizuje sesję poprzez wywołanie metody `StartGame()`, określając tryb działania (host lub klient) oraz parametry sesji, takie jak jej nazwa czy liczba graczy. W modelu host–client jeden z graczy pełni rolę hosta, który tworzy sesję i zarządza jej przebiegiem, natomiast pozostali gracze dołączają jako klienci. Po nawiązaniu połączenia wszystkie instancje uczestniczą w tej samej symulacji sieciowej, co umożliwia wspólną rozgrywkę.

Obsługa zdarzeń sieciowych realizowana jest poprzez implementację interfejsu `INetworkRunnerCallbacks`, który udostępnia zestaw metod wywoływanych w odpowiedzi na zdarzenia takie jak dołączenie lub opuszczenie gracza czy przekazywanie danych wejściowych. Dzięki temu możliwe jest dynamiczne reagowanie na zmiany w stanie sesji, w tym tworzenie obiektów reprezentujących graczy w momencie ich połączenia.

W pierwszym etapie rozwoju systemu gracze reprezentowani byli przez proste obiekty geometryczne dostępne w silniku Unity, w szczególności sześciany. Podejście to jest zgodne z przykładowymi

implementacjami przedstawionymi w dokumentacji, gdzie na początkowym etapie wykorzystywany jest prosty prefab gracza oparty na komponencie NetworkObject. Pozwoliło to na wyeliminowanie dodatkowej złożoności związanej z animacjami oraz systemem renderowania postaci i skoncentrowanie się wyłącznie na mechanice ruchu oraz jej odwzorowaniu w środowisku sieciowym.



Rysunek 1 (Postać gracza w pierwszym etapie rozwoju aplikacji)

Istotnym elementem implementacji sterowania było wykorzystanie mechanizmu przekazywania danych wejściowych udostępnianego przez Photon Fusion. W przeciwieństwie do klasycznego podejścia, w którym zmiany transformacji obiektu są bezpośrednio modyfikowane przez klienta, w Fusion wejście gracza jest najpierw zbierane, a następnie przetwarzane w ramach symulacji sieciowej. Zbieranie danych wejściowych odbywa się w metodzie `OnInput`, implementowanej w ramach interfejsu `INetworkRunnerCallbacks`. Metoda ta jest cyklicznie wywoływana przez `NetworkRunner` i umożliwia przekazanie aktualnego stanu wejścia użytkownika do systemu.

```
1 public void OnInput(NetworkRunner runner, NetworkInput input)
2 {
3     var data = new NetworkInputData();
4     var localInput = PlayerInput.Local;
5
6     if (localInput == null)
7         return;
8
9     data.Direction = new Vector3(localInput.Move.x, 0, localInput.Move.y);
10    data.CameraRotation = localInput.CameraRotation;
11
12    data.Buttons.Set(InputButtons.Jump, localInput.IsJumpPressed);
13    data.Buttons.Set(InputButtons.Interact, localInput.IsInteractPressed);
14
15    input.Set(data);
16 }
```

Rysunek 2 (Zbieranie danych wejściowych gracza)

Zebrane dane wejściowe nie są bezpośrednio stosowane do obiektu gracza, lecz przekazywane do symulacji sieciowej, gdzie są przetwarzane w sposób deterministyczny. Ich wykorzystanie odbywa się w klasie reprezentującej gracza, dziedziczącej po `NetworkBehaviour`, w metodzie `FixedUpdateNetwork()`.

```
1 public override void FixedUpdateNetwork()  
2 {  
3     if (GetInput(out NetworkInputData data))  
4     {  
5         data.Direction.Normalize();  
6         _characterController.Move(data.Direction * speed * Runner.DeltaTime);  
7     }  
8 }
```

Rysunek 3 (Zastosowanie danych wejściowych w symulacji sieciowej)

4.2 Stworzenie postaci finalnej i synchronizacja animacji w sieci

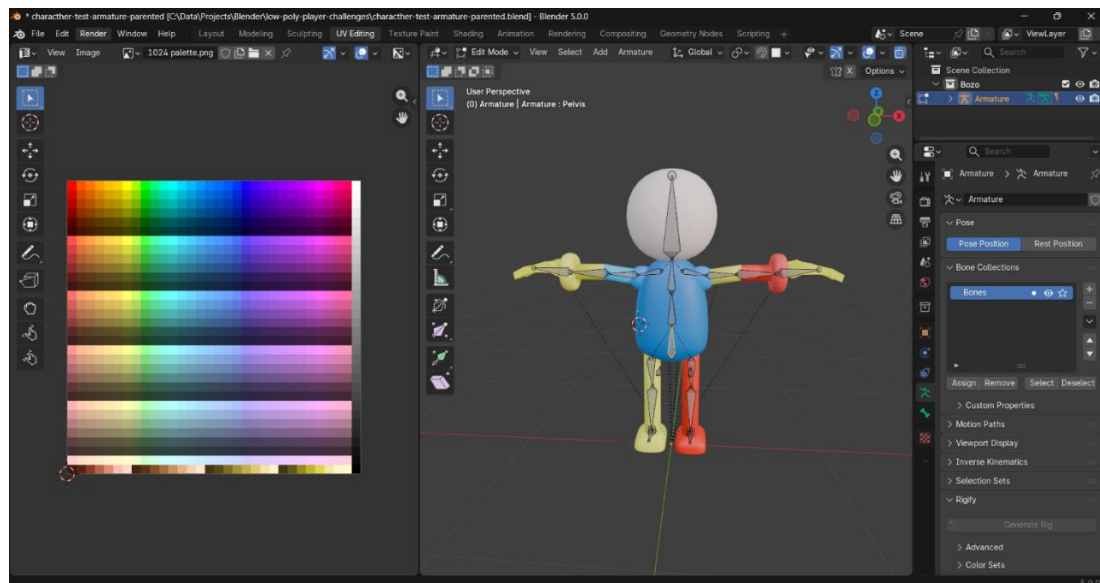
Po zaimplementowaniu podstawowego systemu sterowania oraz synchronizacji ruchu przy użyciu prostych obiektów geometrycznych, kolejnym krokiem było wprowadzenie docelowej postaci gracza wraz z animacjami. W przeciwieństwie do statycznych sześcianów, postać posiadająca model 3D oraz animacje wymaga znacznie bardziej zaawansowanego podejścia do synchronizacji.

Problem ten polegał nie tylko na poprawnym wyświetlaniu ruchu postaci u wszystkich graczy, ale również na synchronizacji stanów animacji, takich jak bieganie, stanie w miejscu czy wykonywanie akcji (np. skok). W środowisku sieciowym konieczne jest zapewnienie, że wszyscy gracze widzą te same animacje w tym samym momencie, mimo opóźnień sieciowych oraz różnic w lokalnym przetwarzaniu danych.

Dodatkowym wyzwaniem było przygotowanie samego modelu postaci w taki sposób, aby umożliwił wykorzystanie animacji w silniku Unity.

W pierwszym etapie rozwiązania problemu przygotowano docelową postać gracza w zewnętrznym narzędziu do modelowania 3D. Model został stworzony samodzielnie w programie Blender, a następnie wyposażony w tzw. rig, czyli strukturę kości umożliwiającą animowanie postaci.

Musimy zauważyć, że zastosowanie riggingu było niezbędne, aby możliwe było przypisanie animacji do modelu oraz ich późniejsze odtwarzanie w silniku Unity. Po przygotowaniu modelu został on zaimportowany do projektu i skonfigurowany jako prefab postaci gracza.



Rysunek 4 (Stworzenie postaci w Blenderze i jej Riggowanie)

Kolejnym krokiem było zintegrowanie modelu z istniejącym systemem sieciowym. Postać gracza została oparta na komponencie `NetworkObject`, co umożliwiło jej identyfikację i synchronizację w środowisku sieciowym. Warstwa wizualna (model i animator) została oddzielona od logiki sieciowej, co pozwoliło na zachowanie czytelnej architektury.

Synchronizacja animacji została zrealizowana poprzez przekazywanie odpowiednich parametrów do komponentu `Animator` na podstawie danych wejściowych gracza. Zamiast bezpośredniego synchronizowania animacji pomiędzy klientami, wykorzystano istniejący mechanizm wejścia (`GetInput()`), który pozwala na deterministyczne odtwarzanie ruchu i stanów postaci.

Przykładowo, kierunek ruchu gracza wykorzystywany był do określenia, czy postać powinna znajdować się w stanie biegu, natomiast brak wejścia powodował przejście do animacji bezczynności.

```
public override void FixedUpdateNetwork()
{
    if (GetInput(out NetworkInputData data))
    {
        data.Direction.Normalize();
        _characterController.Move(data.Direction * speed * Runner.DeltaTime);

        float moveAmount = data.Direction.magnitude;
        _animator.SetFloat("MoveSpeed", moveAmount);
    }
}
```

Rysunek 5 (Aktualizacja parametrów animacji na podstawie danych wejściowych)

Możemy zauważyć, że takie podejście upraszcza synchronizację animacji, ponieważ zamiast przysyłać ich stan przez sieć, wystarczy zsynchronizować dane wejściowe oraz stan logiczny postaci. Wszystkie instancje gry odtwarzają animacje lokalnie na podstawie tych samych danych, co zapewnia spójność wizualną.

4.3 Implementacja systemu lobby publicznych i prywatnych

Do tego momentu rozwoju aplikacji połączenie graczy realizowane było w sposób uproszczony — wszystkie instancje gry dołączały do jednej, z góry zdefiniowanej sesji o stałej nazwie. Podejście to pozwoliło na szybkie przetestowanie mechanizmu synchronizacji ruchu oraz komunikacji sieciowej, jednak znacząco ograniczało funkcjonalność aplikacji.

Widzimy, że w takim modelu użytkownik nie ma kontroli nad tym, z kim rozgrywa sesję, a także nie istnieje możliwość równoległego uruchomienia wielu niezależnych gier. W praktycznej aplikacji wieloosobowej konieczne jest umożliwienie tworzenia wielu sesji jednocześnie oraz zapewnienie mechanizmów pozwalających graczom na świadomy wybór rozgrywki.

W celu rozwiązania powyższego problemu zaimplementowano system lobby oparty na dynamicznie tworzonych sesjach Photon Fusion. Każda sesja reprezentuje osobne lobby, do którego gracze mogą dołączać w zależności od jego dostępności.

Kluczowym elementem rozwiązania było wprowadzenie unikalnych kodów lobby, które pełnią rolę identyfikatora sesji i umożliwiają graczom bezpośrednie dołączanie do wybranej gry. W tym celu opracowano własny mechanizm generowania kodów.

```
1 public string GenerateCode(List<SessionInfo> existingSessions)
2 {
3     char[] buffer = new char[CODE_SIZE];
4
5     for (int attempt = 0; attempt < ATTEMPTS_COUNT; attempt++)
6     {
7         for (int i = 0; i < buffer.Length; i++)
8             buffer[i] = ALPHABET[Random.Range(0, ALPHABET.Length)];
9
10        string code = new string(buffer);
11
12        bool exists = existingSessions.Exists(s =>
13            s.Properties.ContainsKey("code") &&
14            (string)s.Properties["code"] == code);
15
16        if (!exists)
17            return code;
18    }
19
20    Debug.LogError("Failed to generate unique code");
21    return null;
22 }
```

Rysunek 6 (Generowanie unikalnego kodu lobby)

Widzimy, że generowany kod składa się z ograniczonego zestawu znaków, co zmniejsza ryzyko pomyłek podczas jego przepisywania przez użytkownika. Dodatkowo sprawdzana jest jego unikalność względem aktualnie istniejących sesji, co zapobiega kolizjom.

Po wygenerowaniu kodu tworzona jest nowa sesja Photon Fusion. W przeciwieństwie do wcześniejszego podejścia, nazwa sesji nie jest już stała, lecz dynamiczna i powiązana z wygenerowanym kodem. Dodatkowo wykorzystano mechanizm SessionProperties, który pozwala na przechowywanie dodatkowych informacji o lobby.

```
1  var props = new Dictionary<string, SessionProperty>()
2  {
3      { "code", code },
4      { "public", isPublic ? 1 : 0 },
5  };
6
7  var args = new StartGameArgs()
8  {
9      GameMode = GameMode.Host,
10     SessionName = code,
11     PlayerCount = MAX_PLAYERS_COUNT,
12     SessionProperties = props
13 };
14
15 await _runner.StartGame(args);
```

Rysunek 7 (Tworzenie sesji z parametrami lobby)

Dzięki temu każda sesja posiada własny zestaw właściwości, które mogą być wykorzystane do jej identyfikacji oraz filtrowania.

Istotnym elementem systemu było również pobieranie listy dostępnych sesji. Photon Fusion udostępnia mechanizm aktualizacji listy lobby poprzez metodę OnSessionListUpdated, która dostarcza informacje o aktualnie dostępnych grach.

```
1
2  public void OnSessionListUpdated(NetworkRunner runner, List<SessionInfo> sessionList)
3  {
4      _sessions = sessionList;
5  }
```

Rysunek 8 (Aktualizacja listy dostępnych sesji)

Widzimy, że przechowywanie tej listy lokalnie umożliwia późniejsze wyszukiwanie odpowiednich sesji — zarówno losowych, jak i tych odpowiadających konkretnemu kodowi. Dołączanie do lobby prywatnego realizowane jest poprzez przeszukanie listy sesji i odnalezienie tej, której kod odpowiada wartości podanej przez użytkownika. Z kolei dołączanie do lobby publicznego polega na dołączeniu do losowej dostępnej sesji stworzonej przez innych użytkowników.

4.4 Integracja zaproszeń Steam (Steam Invites)

Po zaimplementowaniu systemu lobby publicznych i prywatnych umożliwiającego dołączanie do sesji na podstawie kodów, kolejnym krokiem było rozszerzenie funkcjonalności aplikacji o integrację z platformą Steam. W szczególności istotne było wykorzystanie mechanizmu zaproszeń, który pozwala użytkownikom na szybkie dołączanie do gry bez konieczności ręcznego wprowadzania kodu lobby. Integracja ta jednak wprowadza dodatkowy problem — system zaproszeń Steam operuje na własnych identyfikatorach lobby (CSteamID), które nie są bezpośrednio powiązane z sesjami Photon Fusion. Oznacza to, że w momencie otrzymania zaproszenia aplikacja nie posiada informacji, do której sesji Fusion należy się połączyć. Problem ten można zatem sformułować jako konieczność powiązania dwóch niezależnych systemów: lobby Steam, wykorzystywanego do zaproszeń i sesji Photon Fusion, wykorzystywanej do właściwej rozgrywki.

W celu rozwiązania powyższego problemu zastosowano mechanizm przekazywania kodu lobby Photon Fusion jako dodatkowej danej przechowywanej w lobby Steam. W momencie tworzenia sesji zapisywany jest kod lobby w postaci parametru o nazwie „FusionCode”. Dzięki temu możliwe jest późniejsze odczytanie tej wartości przez gracza, który otrzymał zaproszenie, i wykorzystanie jej do dołączenia do odpowiedniej sesji. Kluczowym elementem implementacji jest komponent nasłuchujący zdarzeń Steam, odpowiedzialny za obsługę zaproszeń.

```
1 private async void OnGameLobbyJoinRequested(GameLobbyJoinRequested_t data)
2 {
3     CSteamID lobby = data.m_steamIDLobby;
4
5     string fusionCode = SteamMatchmaking.GetLobbyData(lobby, "FusionCode");
6
7     if (!string.IsNullOrEmpty(fusionCode))
8     {
9         await JoinFusionSession(fusionCode);
10    }
11    else
12    {
13        _pendingLobby = lobby;
14        SteamMatchmaking.RequestLobbyData(lobby);
15    }
16 }
```

Rysunek 9 (Obsługa zaproszeń Steam i przekazywanie kodu lobby)

Widzimy, że w pierwszej kolejności podejmowana jest próba bezpośredniego odczytania kodu sesji z danych lobby Steam. Jeżeli kod jest dostępny, możliwe jest natychmiastowe rozpoczęcie procesu dołączania do sesji Photon Fusion.

W przypadku, gdy dane lobby nie zostały jeszcze zsynchronizowane, konieczne jest ich jawne pobranie. W tym celu wykorzystywana jest metoda `RequestLobbyData`, a dalsza obsługa realizowana jest w dedykowanym zdarzeniu.

```
1 private async void OnLobbyDataUpdate(LobbyDataUpdate_t data)
2 {
3     CSteamID lobby = new CSteamID(data.m_ulSteamIDLobby);
4
5     if (_pendingLobby != lobby) return;
6
7     string fusionCode = SteamMatchmaking.GetLobbyData(lobby, "FusionCode");
8     if (string.IsNullOrEmpty(fusionCode))
9     {
10         _loadingPanel.Close();
11         return;
12     }
13
14     _pendingLobby = default;
15     await JoinFusionSession(fusionCode);
16 }
```

Rysunek 10 (Obsługa aktualizacji danych lobby Steam)

Po uzyskaniu kodu lobby następuje właściwe połączenie z sesją Photon Fusion.

```
1
2 private async UniTask JoinFusionSession(string fusionCode)
3 {
4     if (!_networkRunner.IsConnected)
5         await _networkRunner.InitAsync();
6
7     await _networkRunner.JoinByCodeAsync(fusionCode);
8 }
```

Rysunek 11 (Dołączanie do sesji Fusion na podstawie kodu)

Możemy zauważyć, że cały proces sprowadza się do przetłumaczenia zaproszenia Steam na operację dołączenia do sesji Fusion. Dzięki temu użytkownik nie musi znać ani wprowadzać kodu lobby — proces ten odbywa się automatycznie poprzez zaproszenie przyjacieli w Steam.

4.5 Implementacja systemu gotowości graczy

W aplikacjach wieloosobowych istotnym elementem organizacji rozgrywki jest zapewnienie, że gra rozpoczyna się dopiero w momencie, gdy wszyscy uczestnicy są na nią przygotowani. W przeciwnym przypadku część graczy mogłaby rozpocząć rozgrywkę wcześniej, co prowadziłoby do niespójnego doświadczenia użytkownika. Widzimy, że problem ten nie należy do najbardziej złożonych pod względem implementacyjnym, jednak ma kluczowe znaczenie dla poprawnego przebiegu rozgrywki. System musi umożliwiać: oznaczenie przez gracza swojej gotowości, synchronizację tego stanu pomiędzy wszystkimi klientami, rozpoczęcie gry dopiero w momencie, gdy wszyscy gracze są gotowi, reagowanie na zmianę stanu (np. cofnięcie gotowości).

Dodatkowym założeniem projektowym było odejście od klasycznego interfejsu użytkownika i zaimplementowanie systemu gotowości jako elementu świata gry.

System gotowości został zrealizowany poprzez wykorzystanie interaktywnych obiektów znajdujących się bezpośrednio w środowisku gry. Gracz może fizycznie podejść do przycisku i wejść z nim w interakcję, co zmienia jego stan gotowości.

Widzimy, że takie podejście zwiększa immersję oraz spójność doświadczenia, ponieważ wszystkie interakcje odbywają się w obrębie świata gry, a nie poprzez zewnętrzny interfejs.

Obsługa interakcji została zaimplementowana w komponencie dziedziczącym po `NetworkBehaviour`, co pozwala na synchronizację działań pomiędzy klientami.

```
1 public void Interact(PlayerRef player, NetworkObject playerObject)
2 {
3     if (!Object.HasStateAuthority)
4         return;
5
6     _lobbyManager.ToggleReady(player);
7 }
```

Rysunek 12 (Obsługa interakcji z przyciskiem gotowości)

Możemy zauważyć, że zmiana stanu gotowości wykonywana jest wyłącznie po stronie obiektu posiadającego State Authority, co zapewnia spójność danych w środowisku sieciowym.

Stan gotowości graczy przechowywany jest w strukturze `NetworkDictionary`, która umożliwia synchronizację danych pomiędzy wszystkimi klientami. Każdy gracz identyfikowany jest przez `PlayerRef`, a przypisana wartość logiczna określa jego aktualny stan gotowości.

Zmiana stanu gotowości realizowana jest poprzez przełączenie wartości w słowniku oraz poinformowanie pozostałych klientów o zmianie.

```
1 public void ToggleReady(PlayerRef player)
2 {
3     if (!HasStateAuthority)
4         return;
5
6     if (!ReadyStates.ContainsKey(player))
7         return;
8
9     bool newValue = !ReadyStates[player];
10    ReadyStates.Set(player, newValue);
11
12    RPC_NotifyPlayerReadyChanged(player, newValue);
13    PlayerReadyChangedEvent?.Invoke(player, newValue);
14
15    CheckAllReady();
16 }
```

Rysunek 13 (Zmiana stanu gotowości gracza)

Widzimy, że po każdej zmianie wywoływana jest metoda sprawdzająca, czy wszyscy gracze są gotowi. Kluczowym elementem systemu jest mechanizm wykrywający moment, w którym wszyscy gracze osiągnęli stan gotowości. W przypadku utraty gotowości przez któregośkolwiek z graczy odliczanie zostaje anulowane. Po spełnieniu warunku gotowości wszystkich graczy rozpoczynane jest odliczanie do startu gry. Wykorzystano do tego mechanizm `TickTimer`, który działa w ramach symulacji sieciowej. Odliczanie jest synchronizowane pomiędzy wszystkimi klientami, a jego stan aktualizowany w metodzie `FixedUpdateNetwork()`. Po jego zakończeniu następuje rozpoczęcie właściwej rozgrywki.

Zaimplementowany system gotowości zapewnia kontrolowany i spójny start gry dla wszystkich uczestników sesji. Widzimy, że zastosowanie sieciowych struktur danych oraz mechanizmów synchronizacji `Photon Fusion` pozwoliło na prostą, a jednocześnie skuteczną realizację tego elementu.

4.6 Obsługa różnorodnych mini gier

W ramach projektu zaprojektowano system minigier w taki sposób, aby umożliwiał ich łatwe rozszerzanie oraz uruchamianie w jednym, spójnym cyklu rozgrywki. Kluczowym założeniem było stworzenie architektury, w której różne typy gier, takie jak minigry czasowe, punktowe czy eliminacyjne, mogą działać w ramach jednego systemu zarządzania, bez konieczności modyfikowania jego podstawowej logiki.

Zamiast tworzyć osobne, niezależne implementacje dla każdego trybu gry, zastosowano podejście oparte o wspólną klasę bazową oraz mechanizm dziedziczenia. Dzięki temu możliwe jest utrzymanie

jednolitego przepływu rozgrywki, przy jednoczesnym zachowaniu dużej elastyczności w implementacji konkretnych mechanik.

Dodatkowo cały system został zaprojektowany tak, aby minigry mogły być uruchamiane przez zewnętrzny manager przebiegu gry, który odpowiada za ich sekwencyjne ładowanie oraz kontrolę cyklu życia. Oznacza to, że każda minigra funkcjonuje jako niezależny moduł, ale jednocześnie wpisuje się w jeden wspólny pipeline rozgrywki.

4.6.1 Klasa bazowa minigry i cykl życia rozgrywki

Podstawą całego systemu jest klasa `MinigameController`, która definiuje wspólny cykl życia każdej minigry. Obejmuje on fazę inicjalizacji, opcjonalne odliczanie, rozpoczęcie gry oraz jej zakończenie. Ważnym elementem tej klasy jest również możliwość blokowania i odblokowywania sterowania gracza w zależności od aktualnej fazy rozgrywki, co zapewnia spójność interakcji w trakcie gry.

```
1 public class MinigameController : NetworkBehaviour
2 {
3     [Networked] public TickTimer CountdownTimer { get; private set; }
4     [Networked] private bool IsCountingDown { get; set; }
5     [Networked] public bool IsGameActive { get; protected set; }
6
7     [SerializeField] protected MinigameDefinitionSO _minigameDefinition;
8     [SerializeField] protected bool _hasCountdown = true;
9
10    public void InitializeMinigame()
11    {
12        if (!HasStateAuthority)
13            return;
14
15        SetPlayersInput(false);
16
17        if (_hasCountdown)
18            StartCountdown();
19        else
20            StartGame();
21    }
22
23    protected virtual void StartGame()
24    {
25        IsGameActive = true;
26        SetPlayersInput(true);
27        RPC_OnGameStart();
28    }
29
30    protected virtual void EndGame()
31    {
32        IsGameActive = false;
33        RPC_OnGameEnd();
34        SetPlayersInput(false);
35    }
36 }
```

Rysunek 14 (Klasa bazowa minigry definiująca wspólny cykl życia rozgrywki)

Dzięki takiej strukturze wszystkie minigry korzystają z jednolitego schematu działania, co upraszcza ich integrację z systemem zarządzania rozgrywką.

4.6.2 Konfiguracja minigier przez ScriptableObject

Każda minigra posiada swoją definicję w postaci ScriptableObject, który pozwala na łatwą konfigurację parametrów bez konieczności ingerencji w kod. W ten sposób możliwe jest definiowanie m.in. identyfikatora gry, sceny, czasu rozpoczęcia oraz typu rozgrywki.

```
1 [CreateAssetMenu(menuName = "Game/Minigame Definition")]
2 public class MinigameDefinitionSO : ScriptableObject
3 {
4     [field: SerializeField] public string Id { get; private set; }
5     [field: SerializeField] public int SceneBuildIndex { get; private set; }
6     [field: SerializeField] public MinigameType GameType { get; private set; }
7     [field: SerializeField] public float GameDuration { get; private set; }
8     [field: SerializeField] public bool IsTimeLimited { get; private set; }
9     [field: SerializeField] public float StartDelay { get; private set; }
10 }
```

Rysunek 15 (Definicja minigry w ScriptableObject)

Takie podejście umożliwia łatwe zarządzanie zestawem minigier oraz ich parametrami bez konieczności zmiany logiki programu.

4.6.3 Rozszerzanie minigier o różne modele rozgrywki

System został zaprojektowany w sposób umożliwiający tworzenie różnych typów minigier poprzez dziedziczenie po klasie bazowej. Każdy typ gry może wprowadzać własne zasady i mechaniki, jednocześnie zachowując wspólny cykl życia. Przykładem takiego rozszerzenia jest system minigier turowych, w których kluczowym elementem jest kontrola kolejności ruchów graczy.

```
1 public abstract class TurnBasedMinigameController : MinigameController
2 {
3     [Networked] public PlayerRef CurrentTurn { get; set; }
4
5     protected List<PlayerRef> _alivePlayers = new();
6
7     protected override void StartGame()
8     {
9         base.StartGame();
10
11         if (!HasStateAuthority) return;
12
13         InitializeAlivePlayers();
14         PickFirstTurn();
15     }
16
17     protected void AdvanceTurn()
18     {
19         int index = _alivePlayers.IndexOf(CurrentTurn);
20         index = (index + 1) % _alivePlayers.Count;
21         CurrentTurn = _alivePlayers[index];
22     }
23 }
```

Rysunek 16 (Rozszerzenie minigry o system turowy)

Przykładem wykorzystania tego systemu jest minigra Mines, w której gracze wykonują ruchy na planszy, a trafienie miny powoduje ich eliminację. Logika ta opiera się na klasie TurnBasedMinigameController, co pozwala skupić się wyłącznie na specyfice rozgrywki, bez konieczności implementowania mechaniki tur od podstaw.

```
1 public class MinesMinigameController : TurnBasedMinigameController
2 {
3     [Inject] private Minefield _minefield;
4
5     protected override void StartGame()
6     {
7         base.StartGame();
8
9         if (!HasStateAuthority)
10             return;
11
12         _minefield.Initialize(this);
13         _minefield.Generate(_alivePlayers.Count);
14     }
15 }
```

Rysunek 17 (Przykład minigry turowej opartej o system bazowy)

Dzięki temu możliwe jest szybkie tworzenie nowych minigier opartych o różne mechaniki, przy zachowaniu spójnej struktury systemu.

4.6.4 Sceny i spawnowanie graczy

Każda minigra znajduje się na osobnej scenie Unity, co pozwala na pełną separację jej zasobów oraz logiki. W każdej scenie znajduje się dedykowany system spawnowania graczy, który odpowiada za ich tworzenie oraz inicjalizację w kontekście danej minigry.

Wszystkie systemy spawnowania dziedziczą po wspólnej klasie bazowej BasePlayerSpawner, co umożliwia zachowanie jednolitego mechanizmu wejścia graczy do gry przy jednoczesnej możliwości jego modyfikacji dla konkretnych trybów rozgrywki.

```
1 public abstract class BasePlayerSpawner : NetworkBehaviour, IPlayerJoined, IPlayerLeft
2 {
3     [Networked, Capacity(6)]
4     public NetworkDictionary<PlayerRef, Player> Players => default;
5
6     [SerializeField] protected NetworkPrefabRef _playerPrefab;
7 }
```

Rysunek 18 (Bazowy system spawnowania graczy)

Przykładem rozszerzenia tej klasy jest MinesPlayerSpawner, który wprowadza dodatkową logikę przypisywania stałych punktów spawn oraz kontrolę ich dostępności.

```

1 public class MinesPlayerSpawner : BasePlayerSpawner
2 {
3     public override void PlayerJoined(PlayerRef player)
4     {
5         Vector3 spawnPosition = _spawnPoints[0].position;
6
7         NetworkObject playerObject = Runner.Spawn(_playerPrefab, spawnPosition, Quaternion.identity, player);
8
9         Players.Add(player, playerObject.GetComponent<Player>());
10    }
11 }

```

Rysunek 19 (Specjalizacja systemu spawnowania dla konkretnej minigry)

Wykorzystane podejście pozwala na tworzenie różnych minigier działających w ramach jednego systemu, przy jednoczesnym zachowaniu wspólnego mechanizmu obsługi graczy oraz cyklu życia rozgrywki.

4.7 Zarządzanie główną pętlą gry

Główna pętla rozgrywki została oparta o klasę `NetworkGameManager`, która pełni rolę centralnego kontrolera całej sesji. Jej zadaniem jest zarządzanie przebiegiem meczu jako sekwencji mini gier, kontrola przełączania scen oraz synchronizacja stanu rozgrywki pomiędzy wszystkimi graczami. W przeciwieństwie do logiki pojedynczych minigier, która działa lokalnie w obrębie jednej sceny, `NetworkGameManager` odpowiada za całość przebiegu rozgrywki od momentu rozpoczęcia meczu aż do jego zakończenia.

Kluczowym elementem systemu jest indeks aktualnej minigry (`CurrentMinigameIndex`), który wskazuje, która gra z przygotowanej listy powinna być aktualnie uruchomiona. Na jego podstawie pobierana jest definicja minigry z `MinigameSceneDatabaseSO`, co pozwala na dynamiczne przełączanie scen bez konieczności zmiany logiki samego kontrolera.

```

1 public void StartMatch()
2 {
3     if (!HasStateAuthority)
4         return;
5
6     int slot = 0;
7     foreach (var player in Runner.ActivePlayers)
8         _playerSlots[slot++] = player;
9
10    ExpectedPlayers = Runner.ActivePlayers.Count();
11    PlayersReady = 0;
12    LoadMinigame(CurrentMinigameIndex);
13 }

```

Rysunek 20 (Inicjalizacja głównej pętli gry)

Przebieg gry realizowany jest w sposób sekwencyjny. Po zakończeniu jednej minigry system przechodzi do kolejnego etapu cyklu, zwiększając indeks aktualnej minigry. Jeżeli w systemie dostępne są kolejne definicje, ładowana jest następna scena przypisana do kolejnej minigry. W przeciwnym przypadku uruchamiany jest proces kończenia całej sesji.

```
1 public void FinishMinigame()
2 {
3     if (!HasStateAuthority)
4         return;
5
6     CurrentMinigameIndex++;
7
8     if (CurrentMinigameIndex >= _database.Count)
9     {
10        StartCoroutine(DelayedFinishMatch());
11        return;
12    }
13
14    PlayersReady = 0;
15    LoadMinigame(CurrentMinigameIndex);
16 }
```

Rysunek 21 (Przejsie pomiędzy minigrami)

Proces przełączania pomiędzy minigramami opiera się na systemie scen Unity zintegrowanym z Fusion. Jak było opisano w poprzednim rozdziale, każda minigra posiada przypisaną własną scenę, określoną w swojej definicji MinigameDefinitionSO. W trakcie zmiany minigry ładowana jest również scena pośrednia (intermission), która pełni funkcję bufora pomiędzy kolejnymi etapami rozgrywki. Dzięki temu możliwe jest płynne przejście pomiędzy scenami bez nagłych zmian kontekstu gry.

Po załadowaniu nowej sceny system oczekuje na inicjalizację kontrolera minigry. Dopiero gdy zostanie on wykryty, uruchamiana jest procedura inicjalizacji, która rozpoczyna cykl danej minigry i podłącza jej zakończenie do głównego kontrolera.

```
1 private async void LoadMinigameScene(int index)
2 {
3     var def = _database.GetByIndex(index);
4     if (def == null) return;
5
6     await Runner.LoadScene(SceneRef.FromIndex(def.SceneBuildIndex));
7
8     if (HasStateAuthority)
9         StartCoroutine(WaitForMinigameAndStart());
10 }
```

Rysunek 22 (Ładowanie i inicjalizacja minigry)

Dzięki takiej strukturze NetworkGameManager nie jest zależny od konkretnego typu minigry ani jej implementacji. Odpowiada wyłącznie za kontrolę przebiegu całego meczu oraz przełączanie pomiędzy kolejnymi etapami rozgrywki, co umożliwia łatwe rozszerzanie systemu o nowe minigry bez ingerencji w jego rdzeń.

4.8 Tabela liderów i wyniki

System wyników w grze został zaprojektowany jako warstwa oddzielona od logiki pojedynczych minigier, dzięki czemu każda z nich może w spójny sposób przekazywać informacje o osiągnięciach graczy do jednego, centralnego miejsca. Rolę tej warstwy pełni właśnie NetworkGameManager, który utrzymuje globalny stan punktacji przez całą sesję rozgrywki.

Podstawą tego jest sieciowa tablica GlobalScores, która przechowuje wyniki przypisane do poszczególnych slotów graczy. Każdy gracz w trakcie rozpoczęcia meczu zostaje przypisany do konkretnego indeksu, co pozwala na jednoznaczne mapowanie wyników niezależnie od zmiany scen czy rodzaju minigry. Dzięki temu system nie operuje bezpośrednio na obiektach graczy, lecz na ich stabilnych identyfikatorach w ramach sesji.

Każda minigra w trakcie swojego cyklu życia przekazuje informacje o wynikach w momencie zakończenia rozgrywki. Dzięki temu NetworkGameManager nie musi znać szczegółów implementacyjnych poszczególnych trybów, a jedynie przyjmuje ustandaryzowane dane wejściowe.

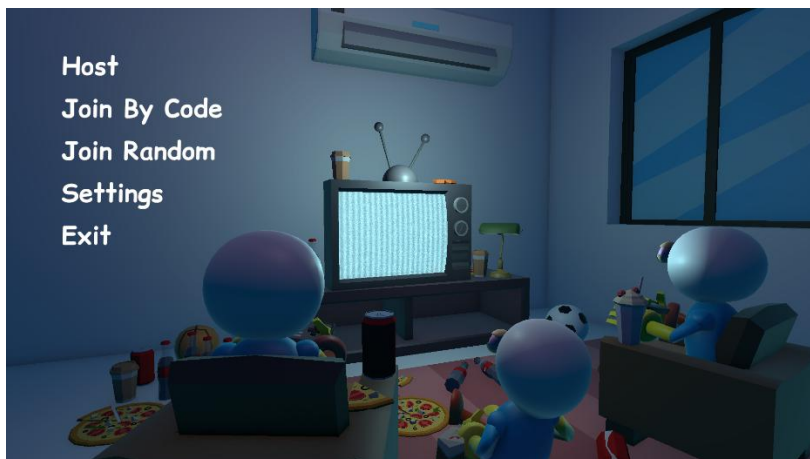
Po zakończeniu całego meczu system dokonuje finalnego zestawienia wyników. Dane z GlobalScores są przetwarzane i sortowane w celu określenia końcowej kolejności graczy. Na tym etapie tworzona jest już finalna tabela wyników, która reprezentuje pełny przebieg rywalizacji.

Rozdział 5

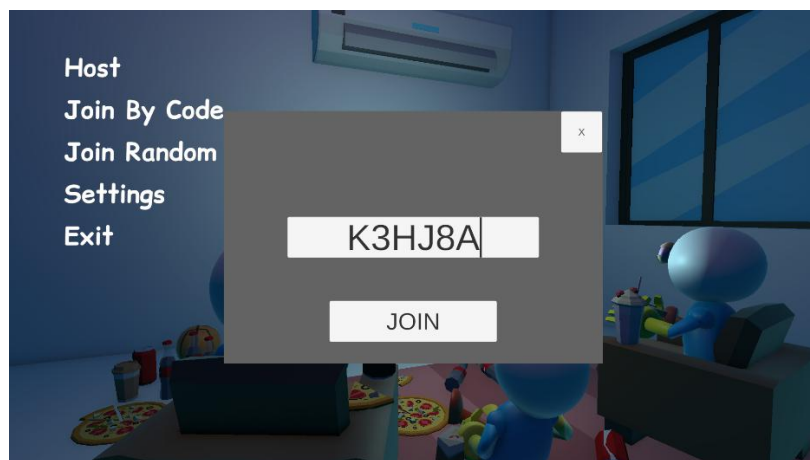
Opis aplikacji

Zaimplementowana aplikacja umożliwia przejście przez pełny cykl rozgrywki wieloosobowej – od wyboru trybu gry, przez lobby i kolejne minigry, aż po prezentację wyników. Poniżej przedstawiono najważniejsze elementy interfejsu oraz przebieg działania systemu z perspektywy użytkownika.

Pierwszym etapem jest menu główne, w którym użytkownik może zdecydować, czy chce utworzyć nowe lobby, czy dołączyć do istniejącej sesji. Aplikacja umożliwia dołączenie do losowego lobby publicznego lub do lobby prywatnego poprzez wprowadzenie odpowiedniego kodu. Takie rozwiązanie pozwala zarówno na szybkie rozpoczęcie gry z przypadkowymi graczami, jak i na organizowanie rozgrywek w zamkniętym gronie znajomych.

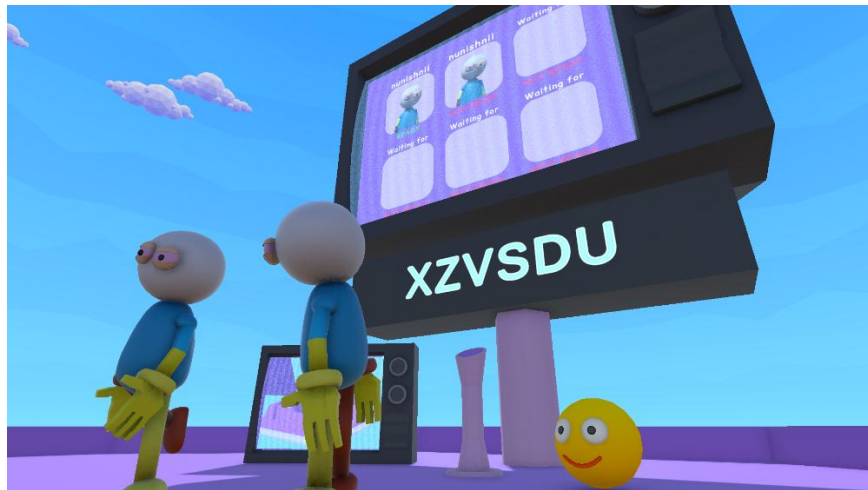


Rysunek 23 (Menu główne z możliwością tworzenia lobby lub dołączania)



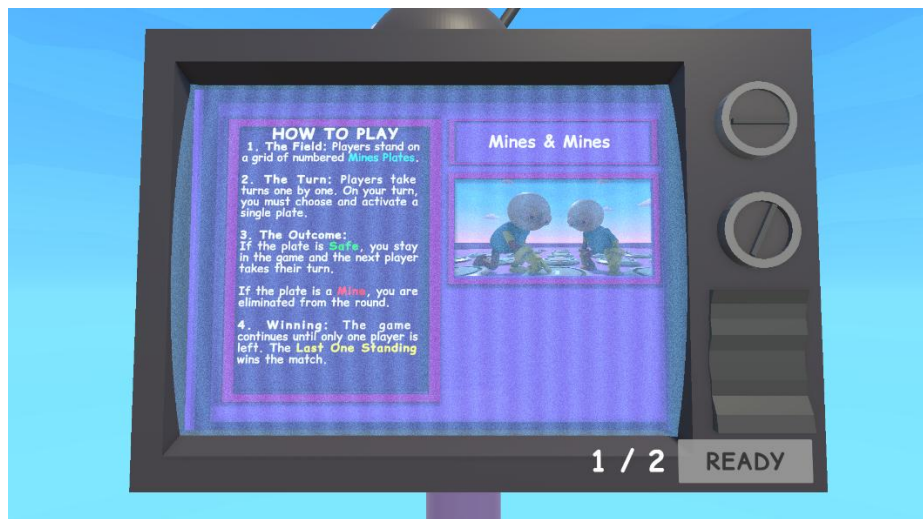
Rysunek 24 (Wprowadzanie kodu lobby prywatnego)

Po dołączeniu do sesji użytkownik trafia do lobby, w którym widoczna jest lista graczy oczekujących na rozpoczęcie rozgrywki. Każdy uczestnik może oznaczyć swoją gotowość, a aktualny stan wszystkich graczy jest na bieżąco wyświetlany. Rozpoczęcie gry następuje dopiero po potwierdzeniu gotowości przez wszystkich uczestników, co zapewnia zsynchronizowany start.



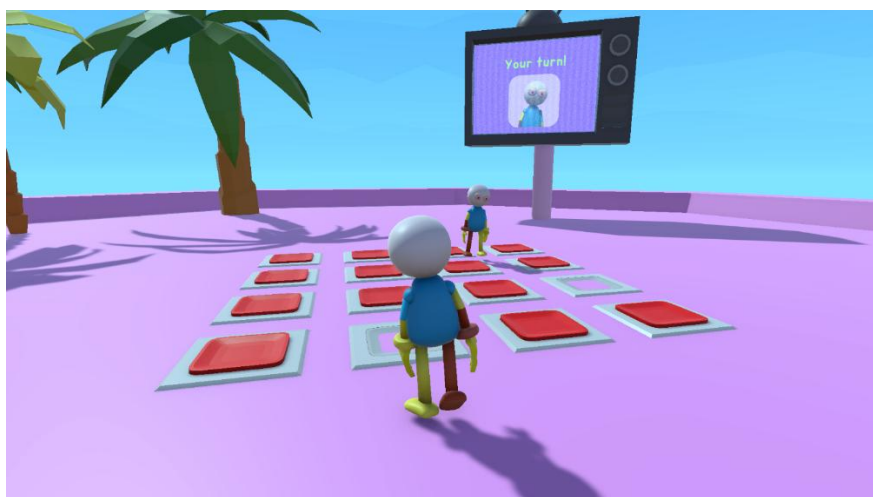
Rysunek 25 (Lobby gry z ekranem stanu graczy)

Po rozpoczęciu rozgrywki system przechodzi do ekranu opisu minigry. W tym miejscu prezentowane są podstawowe informacje, takie jak nazwa minigry, jej zasady oraz obraz poglądowy. Ekran ten pojawia się przed każdą minigrą i stanowi element spójnej pętli rozgrywki, przygotowując graczy do kolejnego etapu. Dodatkowo, podobnie jak w lobby, wszyscy uczestnicy muszą potwierdzić swoją gotowość, aby możliwe było rozpoczęcie danej minigry.



Rysunek 26 (Ekran opisu minigry – nazwa, zasady i grafika)

Jedną z zaimplementowanych minigier jest gra typu „Mines”. Rozgrywka polega na wybieraniu pól planszy, pod którymi mogą znajdować się miny. Gracze wykonują ruchy naprzemiennie, a trafienie na minę skutkuje eliminacją z dalszej rozgrywki. Istotnym elementem tej minigry jest skalowanie planszy w zależności od liczby graczy – przykładowo dla dwóch graczy wykorzystywana jest plansza 3×3, natomiast dla trzech graczy 4×4. Dzięki temu rozgrywka zachowuje odpowiedni poziom trudności i dynamiki niezależnie od liczby uczestników.



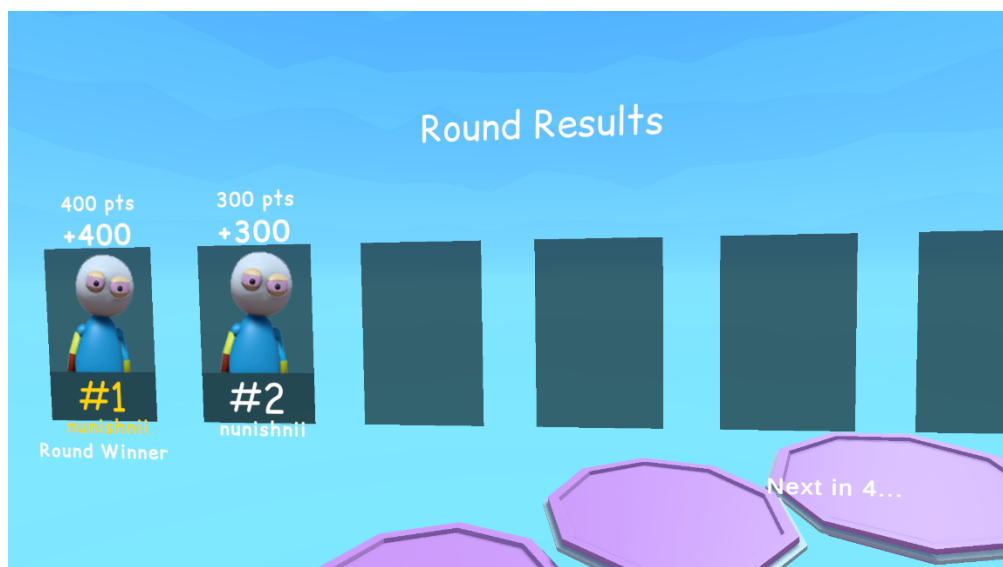
Rysunek 27 (Minigra „Mines” – plansza gry)

W celu zaprezentowania możliwości systemu w zakresie różnorodności minigier zaimplementowano również prostą grę wyścigową. Jej zasada polega na tym, że gracze rywalizują, kto jako pierwszy dotrze do mety. Minigra ta wykorzystuje inny typ mechaniki niż „Mines”, co pokazuje elastyczność systemu i możliwość tworzenia zróżnicowanych doświadczeń w ramach jednej platformy.



Rysunek 28 (Minigra wyścigowa – widok toru i graczy)

Po zakończeniu minigry wyświetlana jest tabela wyników, prezentująca osiągnięcia wszystkich uczestników. Pozwala ona na bieżąco śledzić postępy oraz porównywać rezultaty pomiędzy graczami w trakcie całej sesji.



Rysunek 29 (Tabela liderów – wyniki graczy)

Dodatkowo aplikacja została zintegrowana z platformą Steam, co umożliwia korzystanie z systemu zaproszeń (Steam Invite). Dzięki temu użytkownicy mogą w prosty sposób zapraszać swoich znajomych bezpośrednio z poziomu nakładki Steam (Steam Overlay), bez konieczności ręcznego wprowadzania kodu lobby. Funkcjonalność ta znacząco usprawnia proces organizowania wspólnej rozgrywki.



Rysunek 30 (Steam Overlay – wysyłanie zaproszenia do gry)

Przedstawione elementy pokazują, że aplikacja realizuje pełny cykl rozgrywki wieloosobowej, zapewniając spójne przejścia pomiędzy kolejnymi etapami oraz możliwość implementacji różnorodnych minigier w ramach jednego systemu.

Rozdział 6

Możliwe kierunki rozwoju aplikacji

Zaimplementowana aplikacja stanowi w pełni działającą platformę minigier wieloosobowych, jednak jej architektura została zaprojektowana w sposób umożliwiający dalszy rozwój oraz rozbudowę o nowe funkcjonalności. Poniżej przedstawiono wybrane kierunki, w których system może być rozwijany.

Jednym z najważniejszych obszarów rozwoju jest rozszerzenie zestawu dostępnych minigier. Dzięki zastosowaniu uniwersalnej architektury możliwe jest stosunkowo łatwe dodawanie nowych typów rozgrywek o różnych zasadach i mechanikach. Wprowadzenie większej liczby minigier zwiększyłoby różnorodność oraz atrakcyjność aplikacji, a także wydłużyło czas zaangażowania użytkowników.

Kolejnym kierunkiem jest rozbudowa systemu rozgrywki o dodatkowe tryby, takie jak tryb rankingowy, w którym wyniki graczy byłyby zapisywane i porównywane globalnie. Możliwe jest również wprowadzenie systemu sezonów lub lig, co dodatkowo motywowałoby użytkowników do regularnej gry.

Istotnym usprawnieniem może być rozwój systemu lobby oraz zarządzania sesją. Można wprowadzić dodatkowe opcje konfiguracyjne, takie jak wybór konkretnych minigier, ustawienia długości rozgrywki czy ograniczenia liczby graczy. Rozszerzenie funkcji społecznościowych, na przykład poprzez czat głosowy lub tekstowy, mogłoby znacząco poprawić komfort komunikacji pomiędzy uczestnikami.

W kontekście integracji z platformą Steam możliwe jest rozszerzenie wykorzystania jej funkcjonalności o system osiągnięć, statystyk graczy czy zapis postępów w chmurze. Takie rozwiązania zwiększają zaangażowanie użytkowników oraz pozwalają na długoterminowe śledzenie ich aktywności.

Kolejnym kierunkiem rozwoju jest poprawa warstwy wizualnej oraz interfejsu użytkownika. Możliwe jest wprowadzenie bardziej zaawansowanych animacji, efektów wizualnych oraz dopracowanie wyglądu interfejsu, co przełoży się na lepsze doświadczenie użytkownika.

Nie bez znaczenia jest również optymalizacja wydajności oraz stabilności systemu sieciowego. W przyszłości można rozważyć implementację bardziej zaawansowanych mechanizmów synchronizacji, poprawę obsługi błędów sieciowych oraz dostosowanie aplikacji do działania przy większej liczbie graczy.

Podsumowując, aplikacja posiada solidne podstawy umożliwiające dalszy rozwój zarówno pod względem funkcjonalnym, jak i technicznym. Dzięki modularnej architekturze możliwe jest stopniowe rozszerzanie systemu bez konieczności wprowadzania istotnych zmian w jego podstawowej strukturze.

Rozdział 7

Bibliografia

1. Unity, 2026. *Unity 6.4 User Manual*, Unity Documentation. Dostęp online: [Unity Documentation](#)
2. Photon, 2026, *Fusion 2 Introduction*, Photon Engine. Dostęp online: [Photon Fusion 2 Introduction](#)
3. Valve, 2026, *Steamworks Documentation*, SteamWorks. Dostęp online: [Steam SDK Documentation](#)
4. Ian Griffiths, 2023, *C# 10. Programowanie. Tworzenie aplikacji Windows, internetowych i biurowych*. Dostęp online: [C# 10. Programowanie. Tworzenie aplikacji Windows, internetowych i biurowych](#)
5. Mohamed Esam, 2024, *Mastering Unity Game Development with C#. Harness the full potential of Unity 2022 game development using C#*. Dostęp online: [Mastering Unity Game Development with C#. Harness the full potential of Unity 2022 game development using C#](#)
6. Joe Hocking, 2022, *Unity in Action, Third Edition: Multiplatform Game Development in C#*. Dostęp online: [Unity in Action, Third Edition: Multiplatform Game Development in C#](#)
7. Jeremy Gibson Bond, 2018, *Projektowanie gier przy użyciu środowiska Unity i języka C#. Od pomysłu do gotowej gry. Wydanie II*. Dostęp online: [Projektowanie gier przy użyciu środowiska Unity i języka C#. Od pomysłu do gotowej gry. Wydanie II](#)

Spis rysunków

Rysunek 1 (Postać gracza w pierwszym etapie rozwoju aplikacji)	12
Rysunek 2 (Zbieranie danych wejściowych gracza)	12
Rysunek 3 (Zastosowanie danych wejściowych w symulacji sieciowej)	13
Rysunek 4 (Stworzenie postaci w Blenderze i jej Riggowanie)	14
Rysunek 5 (Aktualizacja parametrów animacji na podstawie danych wejściowych).....	14
Rysunek 6 (Generowanie unikalnego kodu lobby)	15
Rysunek 7 (Tworzenie sesji z parametrami lobby)	16
Rysunek 8 (Aktualizacja listy dostępnych sesji).....	16
Rysunek 9 (Obsługa zaproszeń Steam i przekazywanie kodu lobby).....	17
Rysunek 10 (Obsługa aktualizacji danych lobby Steam).....	18
Rysunek 11 (Dołączanie do sesji Fusion na podstawie kodu)	18
Rysunek 12 (Obsługa interakcji z przyciskiem gotowości)	19
Rysunek 13 (Zmiana stanu gotowości gracza).....	20
Rysunek 14 (Klasa bazowa minigry definiująca wspólny cykl życia rozgrywki)	21
Rysunek 15 (Definicja minigry w ScriptableObject)	22
Rysunek 16 (Rozszerzenie minigry o system turowy).....	22
Rysunek 17 (Przykład minigry turowej opartej o system bazowy)	23
Rysunek 18 (Bazowy system spawnowania graczy)	23
Rysunek 19 (Specjalizacja systemu spawnowania dla konkretnej minigry)	24
Rysunek 20 (Inicjalizacja głównej pętli gry)	24
Rysunek 21 (Przejście pomiędzy minigrami)	25
Rysunek 22 (Ładowanie i inicjalizacja minigry)	25
Rysunek 23 (Menu główne z możliwością tworzenia lobby lub dołączania)	27
Rysunek 24 (Wprowadzanie kodu lobby prywatnego)	27
Rysunek 25 (Lobby gry z ekranem stanu graczy).....	28
Rysunek 26 (Ekran opisu minigry – nazwa, zasady i grafika).....	28
Rysunek 27 (Minigra „Mines” – plansza gry	29
Rysunek 28 (Minigra wyścigowa – widok toru i graczy)	29
Rysunek 29 (Tabela liderów – wyniki graczy).....	30
Rysunek 30 (Steam Overlay – wysyłanie zaproszenia do gry)	30